

Plateforme centralisée d'analyse des logs des frontaux HTTP en temps réel dans un milieu virtualisé

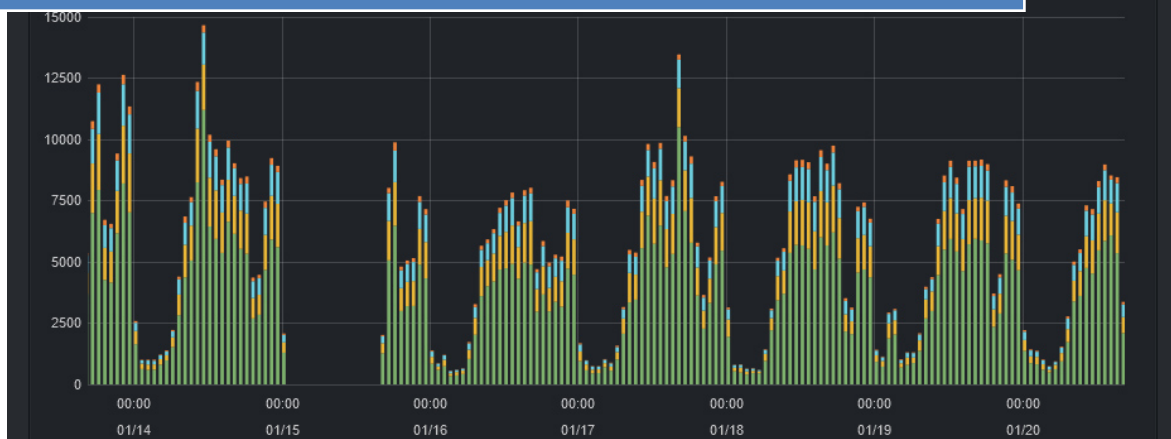


Table des matières

Index des figures.....	3
Introduction.....	4
1 Les débuts d'Internet, l'air du web « 1.0 »	6
1.1 Architecture web minimaliste.....	6
1.2 Définition d'un « log »	6
1.3 Analyser les logs	7
1.4 Montée en charge	8
2 La virtualisation.....	8
2.1 Présentation.....	8
2.2 Avantages & inconvénients.....	10
3 L'Internet moderne : fort trafic & qualité de service	11
3.1 Architecture web haute disponibilité avec tolérance des pannes	11
3.2 Les nouvelles problématiques.....	12
4 Plateforme d'analyse de log centralisée	12
4.1 Présentation.....	12
4.2 Plus-value.....	13
4.3 Architecture globale	14
4.4 Emission des logs : Apache & autres	14
4.4.1 Les fondamentaux	14
4.4.2 Problématique liée au load balancer	15
4.5 Transport des logs : Rsyslog	15
4.6 Formatage & Enrichissement des logs : Logstash	16
4.6.1 Présentation	16
4.6.2 Les plug-ins d'entrées : Input	17
4.6.3 Les plug-ins de traitements : Filters.....	18
4.6.4 Les plug-ins de sorties : Output	20
4.7 Stockage des logs : Elasticsearch	20
4.7.1 Présentation	20
4.7.2 Problématiques d'indexation de logs dans un index « full texte ».....	20
4.8 Analyse des logs temps réel : Kibana.....	21
4.8.1 Présentation	21
4.8.2 Tableau de bord & analyse.....	22
Conclusion	27

Index des figures

Figure 1 : Architecture web « 1.0 ».....	6
Figure 2 : Cycle de vie d'un log	7
Figure 3 : Hyperviseur de type 1.....	9
Figure 4 : Hyperviseur de type 2.....	9
Figure 5 : Architecture web haute disponibilité avec tolérance des pannes	11
Figure 6 : Chaîne logicielle de la plateforme d'analyse de log en temps réel	13
Figure 7 : Architecture web et plateforme d'analyse de log centralisée en temps réel.....	14
Figure 8 : Principe de fonctionnement d'Rsyslog	16
Figure 9 : Architecture Logstash à scalabilité horizontale	17
Figure 10 : Visualisation des logs d'accès Apache sur la plateforme Kibana	24
Figure 11 : Top 10 des valeurs d'une metric	25
Figure 12 : Détail d'un log d'accès Apache sur la plateforme Kibana 1 de 2	25
Figure 13 : Détail d'un log d'accès Apache sur la plateforme Kibana 2 de 2	26

Introduction

Les débuts du Word Wide Web, plus couramment appelé « web » remontent au 13 novembre 1990. Ce sont deux ingénieurs – Tim BERNERS-Lee et Robert CAILLIAU – du CERN de Genève en Suisse qui ont mis au point le premier serveur web de l'histoire¹. A cette époque, les contenus sont principalement composés de documents textes et d'images. Ce sont essentiellement des personnes aguerries qui consultent Internet : scientifiques ainsi qu'universitaires. La consommation de ces services est dite « passive » : les internautes consomment l'information mise à disposition d'un producteur : l'éditeur du site. On parle de web « read-only » ou encore « web 1.0 ». En 1999, soit environ dix ans après l'apparition d'Internet moins de 10% des Français sont connectés au réseau des réseaux².

La chute des prix des micro-ordinateurs (en 1990 le prix moyen d'un PC était de 7 869 €³ contre 1 043 €³ dix ans plus tard) a contribué à diversifier le public d'Internet. C'est au cours de la première décennie du XXIème siècle qu'Internet initie sa professionnalisation. Les enjeux changent, le web devient « social » et collaboratif, c'est l'air du « web 2.0 ». Les nouvelles possibilités offertes par ce canal de communication sont immenses et de multiples formes : e-commerce, divertissements, services, informations, etc. Le nombre d'utilisateurs d'Internet croît de façon exponentielle. Au cours de l'année 2006, plus de 50% de la population Française est connectée à Internet². La majorité des visiteurs ne sont plus des techniciens avertis mais des utilisateurs lambda en quête de services ou d'informations. Face à un public de plus en plus nombreux, l'architecture des sites web change afin de pouvoir supporter une audience grandissante d'année en année.

Avec la démocratisation ces dernières années du (très) haut-débit, des smartphones et des tablettes, Internet n'est plus limité au seul support des ordinateurs « Desktop ». Le développement de services web se complexifie : sites web « multi-devices » qui imposent le développement et l'exploitation d'un même service à destination de plusieurs plateformes (« Desktop », iOS, Android, Windows Phone, consoles de jeux, TV connectée, bornes interactives, etc.) aux caractéristiques différentes. Internet devient interactif. On parle de réalité augmentée (appareil photo/vidéo, tactile, GPS). Certains experts parlent de « web 4.0 »⁴. Afin que l'expérience utilisateur soit fluide et réactive, le temps de réponse des services web – portail web, API REST / SOAP – doit être au plus bas.

L'achat en ligne – de biens et/ou services – s'est progressivement introduit dans les mœurs. En 2012, 69% des français achètent à distance, ce pourcentage important est principalement issu du e-commerce. Ce secteur pèse en France 45 milliards d'euros et est vecteur de 75 000 emplois⁵. Les dommages, financiers et/ou image de marque, en cas de défaillance technique grandissent avec les années. En 2010, le manque à gagner d'Amazon, Google ou eBay est estimé à respectivement à 1084\$, 929\$, 290\$ par seconde d'indisponibilité de leurs services⁶.

¹ <http://info.cern.ch/hypertext/WWW/TheProject.html>

² perspective.usherbrooke.ca/bilan/servlet/BMTendanceStatPays?codeStat=IT.NET.USER.P2&codePays=FRA

³ <http://www.volle.com/ENSPTT/primicro.htm>

⁴ <http://aurorecapriles.wordpress.com/2013/11/26/le-web-3-0-web-symbiotique/>

⁵ http://www.fevad.com/uploads/files/Publications/Chiffres_Cles_2013%281%29.pdf

⁶ <http://www.incomediary.com/20-websites-making-the-most-money>

Ces nouvelles problématiques nécessitent que les équipes techniques se dotent de nouveaux outils afin d'offrir aux Internauteurs un service toujours plus performant, sécurisé et fiable. Ceci dans le but de pouvoir identifier au plus vite l'incident. Afin de faciliter la compréhension du dysfonctionnement, un grand nombre d'informations techniques doivent être collectées : parcours utilisateur, compte utilisateur, adresse IP, type de périphérique, type de navigateur Internet, etc. Celles-ci sont difficiles à obtenir avec le niveau de détail attendu par une personne technique. Ces nouveaux outils donnent également des indicateurs sur la qualité de service de l'application (temps de réponse des pages).

Un service web fiable – Tolérance aux pannes et haute disponibilité⁷ – repose sur une architecture serveurs redondante. La virtualisation des serveurs apporte cette redondance sans multiplier le nombre de machines physiques. Elle réduit les coûts énergétiques tout en simplifiant l'administration. Elle permet, entre autre, le déploiement à grande échelle d'une configuration sur différentes machines physiques, c'est le Cloud Computing.

Cet article traite d'une part, de la solution de virtualisation d'infrastructure serveurs via la plateforme VMware vSphere / vCenter⁸ et d'autre part, de la plateforme open source d'analyse de logs des frontaux web en temps réel basée sur Rsyslog⁹ (extension du protocole basique Syslog¹⁰), Logstash¹¹, Elasticsearch¹² et Kibana¹³.

⁷ http://fr.wikipedia.org/wiki/Haute_disponibilit%C3%A9

⁸ <http://www.vmware.com/fr/products/vsphere>

⁹ <http://www.rsyslog.com/>

¹⁰ <http://tools.ietf.org/html/rfc5424>

¹¹ <http://logstash.net/>

¹² <http://www.elasticsearch.org/>

¹³ <http://www.elasticsearch.org/overview/kibana/>

1 Les débuts d'Internet, l'air du web « 1.0 »

1.1 Architecture web minimaliste

A l'origine, Internet est utilisé par une minorité de personnes. En 2004, moins de 40% des Français sont connectés à Internet². Il s'agit essentiellement d'un public d'experts et d'initiés. Une majorité de personnes se connecte à Internet par le biais de l'accès par ligne commutée – Dial-up. Ce type de ligne souffre d'instabilité ainsi que de faible débit : maximum 7Ko/sec. Le chargement d'une page web peut prendre plusieurs minutes. Dans ces conditions, le temps de traitement du serveur web pour fournir la page est négligeable vis-à-vis du temps d'acheminement. Il n'existe pas de réel écosystème bâti autour d'Internet, les sociétés ne s'en servent pas encore comme « vitrine » et les emplois autour du web sont encore marginaux. Le e-commerce en est à ses balbutiements. C'est Amazon, en véritable pionnier, qui ouvre la voie en démarrant son activité en juillet 1995¹⁴. Face au manque de sécurité des paiements électroniques sur Internet, le public est réticent à effectuer des achats sur ce canal privilégiant ainsi le minitel.

Compte tenu des faibles performances de calculs des machines de l'époque, l'architecture serveur privilégiée consiste à déployer un serveur physique par service fourni : web, mail, base de données, etc.

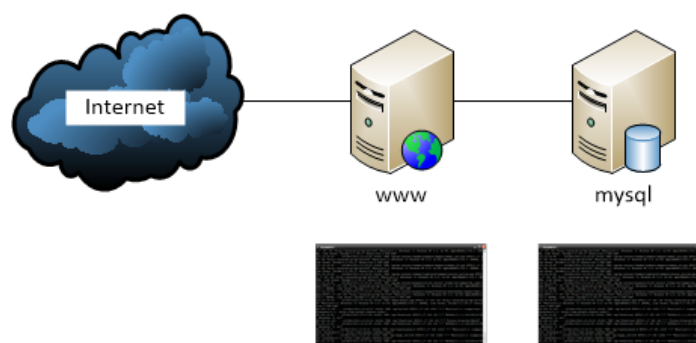


Figure 1 : Architecture web « 1.0 »

La gestion des incidents repose principalement sur des sauvegardes effectuées à un instant « t », plus généralement la nuit. Dans le meilleur des cas, la sauvegarde est stockée manuellement sur différents sites géographiques. Lorsqu'un incident survient, le service est interrompu le temps que les équipes techniques réparent le/les serveurs. Dans le cas d'un incident sévère, il faut réinitialiser les données de production avec celles de la dernière sauvegarde. Cela engendre la perte des informations créées depuis la dernière sauvegarde, qui peut être plus ou moins importante en fonction de la fréquence des enregistrements.

1.2 Définition d'un « log »

Le présent article traite uniquement des logs de type « événement ». Les logs binaires – MySQL, Oracle, etc. – ne sont pas traités car destinés à d'autres usages comme la réplication.

Un log est un message / événement daté, journalisé dans un fichier. Chaque événement représente une ligne dans le fichier. Une ligne contient toutes les caractéristiques de l'événement :

¹⁴ www.actualitte.com/reportages/l-ebook-a-40-ans-1995-amazon-pionnier-du-cybercommerce-1476.htm

adresse IP, url, user agent, code réponse, etc. Les lignes sont indépendantes les unes des autres. Les nouveaux événements sont ajoutés en fin de fichier. L'unique manipulation réalisée post écriture est l'archivage des logs afin de limiter la taille du fichier en cours d'écriture, appelée logrotate¹⁵. Il est également possible de hiérarchiser les logs des serveurs web suivant la date et l'heure de l'évènement grâce à cronolog¹⁶. La nature des événements est diverse. Elle varie en fonction des applications qui génèrent les logs. Voici les principaux types d'évènements :

- Opération en erreur : connexion impossible à la base de données.
- Opération réussie : mise à jour du profil d'un utilisateur.
- Information : pourcentage d'avancement d'un traitement.
- Action utilisateur : clic sur une bannière, ouverture d'un email, accès à une ressource.

Le cycle de vie d'un log est composé des 5 états ci-dessous :

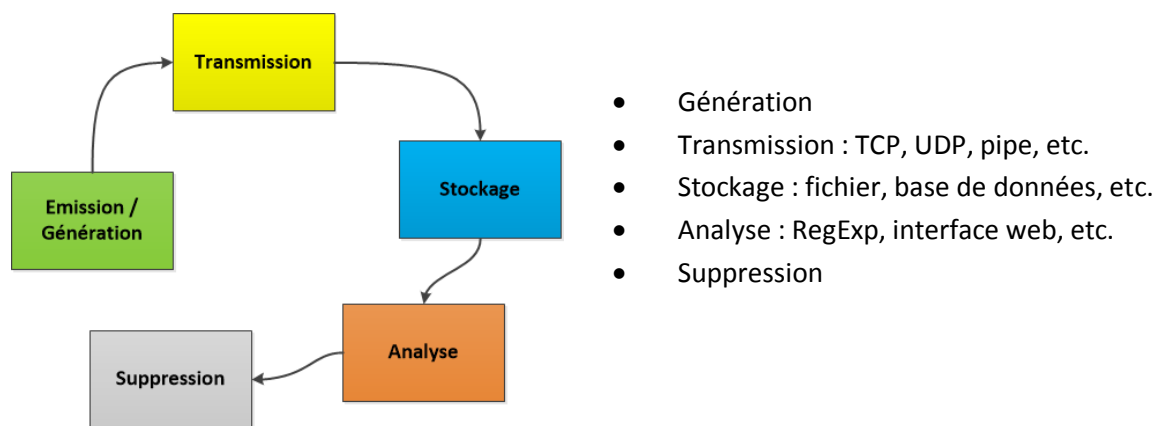


Figure 2 : Cycle de vie d'un log

La volumétrie des logs peut très vite saturer l'espace disque du système. Si les logs sont sur la même partition que l'application et/ou la partition racine du système, le service sera interrompu. Le serveur web journalise l'ensemble des accès aux différents éléments qui composent une page web : document HTML, CSS, JavaScript, images, etc. Dans l'exemple d'une page composée de 30 éléments, avec une moyenne de 300 octets par événement (variable en fonction de l'url et du user agent) l'appel de la page par un seul client génère environ 8.8ko de log.

1.3 Analyser les logs

La nature hétérogène des logs rend complexe la mise en place d'une plateforme centralisée d'analyse de log. En effet, les valeurs mesurées dépendent du système analysé : accès web, erreur web, requêtes SQL lentes, etc. De plus, développer une telle plateforme dans un environnement non virtualisé représente un coût non négligeable. Celui par exemple d'une machine physique inexploitée pour la production. Etant donné que chaque serveur qui gère un service possède l'ensemble des logs du service, il est possible de se connecter en SSH aux différents serveurs afin de pouvoir enquêter manuellement sur la panne : commande tail, awk, RegExp, etc. Le filtrage, code d'erreur 404, ainsi que l'agrégation d'information (top 10 des adresses IP qui ont consultés le plus de pages) est complexe. Il est également difficile d'analyser l'historique des événements à cause de la rotation des

¹⁵ <http://www.thegeekstuff.com/2010/07/logrotate-examples/>

¹⁶ <http://cronolog.org/>

logs. En effet, les anciens logs sont soit archivés, soit répartis dans plusieurs dossiers. De plus, sans rotation, la navigation est difficile car le fichier peut faire plusieurs giga octets. Enfin, la création de tableaux de bord « maison » automatisés requiert des développements personnalisés et complexes. La connexion en SSH va à l'encontre des politiques de sécurité en vigueur dans les grandes entreprises. En effet, les développeurs n'ont pas à avoir accès aux serveurs de production, seuls les administrateurs systèmes et réseaux ont cette possibilité mais ces derniers ont une faible connaissance de l'application, par conséquent la lecture des logs est difficile.

1.4 Montée en charge

Afin de répondre au nombre croissant de visiteurs, il existe deux méthodes pour que le système d'information puisse tenir la charge sans dégrader la qualité de service :

- Scalabilité verticale : cette pratique consiste à utiliser des machines de plus en plus puissantes (processeur plus puissant avec plus de mémoire, etc). Cette technique ne permet pas d'absorber les pics de trafic. Cette architecture est faiblement orientée vers la parallélisation de processus à cause du nombre fixe de serveurs.
- Scalabilité horizontale : La charge est répartie sur un ensemble de serveurs (cluster, pool de serveur applicatif). Il est possible d'ajouter ou de retirer facilement des serveurs pour gérer les pics de trafic. En plus de pouvoir gérer la parallélisation, cette architecture est résistante aux pannes. Le service n'est pas paralysé par la perte d'un serveur, les données sont toujours accessibles.

L'évolution de la puissance des machines ainsi que la généralisation des systèmes de cache (fichier, mémoire vive, CDN¹⁷, etc.) contribue à réduire la charge des serveurs. En moyenne, la charge d'un serveur physique est de 15%¹⁸. L'économie d'énergie par rapport à un serveur chargé à 90% n'est pas significative.

2 La virtualisation

2.1 Présentation

Les origines de la virtualisation remontent aux années 1970 grâce au développement du système expérimental CP/CMS au centre scientifique de Cambridge d'IBM en collaboration avec le MIT¹⁹. Le système est composé d'un programme de contrôle, le « CP ». Ce dernier crée et manipule de multiples machines virtuelles indépendantes, les « VMs ». Cette architecture en avance sur son temps, est la source d'inspiration des développements des techniques de virtualisation moderne. La première plateforme de virtualisation sur l'architecture x86 a été mise au point par la société américaine VMware avec la sortie commerciale le 8 février 1999 du produit VMware Workstation.

La virtualisation consiste à faire fonctionner un ou plusieurs systèmes d'exploitation et/ou applications comme un simple logiciel, sur une ou plusieurs machines (hyperviseur de type 1) ou système d'exploitation (hyperviseur de type 2). Un hyperviseur est une fine couche logicielle – OS

¹⁷ CDN : Content Delivery Network - http://fr.wikipedia.org/wiki/Content_delivery_network

¹⁸ <https://www.gandi.net/hosting/iaas/green>

¹⁹ <http://en.wikipedia.org/wiki/CP/CMS>

hôte – située entre la couche matérielle de la machine physique et les systèmes d’exploitation et/ou applications virtuelles – OS invités.

Un hyperviseur de type 1 – bare metal – est un système installé directement sur une plateforme matérielle. C’est un noyau système minimaliste, optimisé pour gérer les accès entre les systèmes d’exploitation invités et l’architecture matérielle physique. On parle de Paravirtualisation lorsque les systèmes d’exploitation invités ont « conscience » d’être virtualisés.

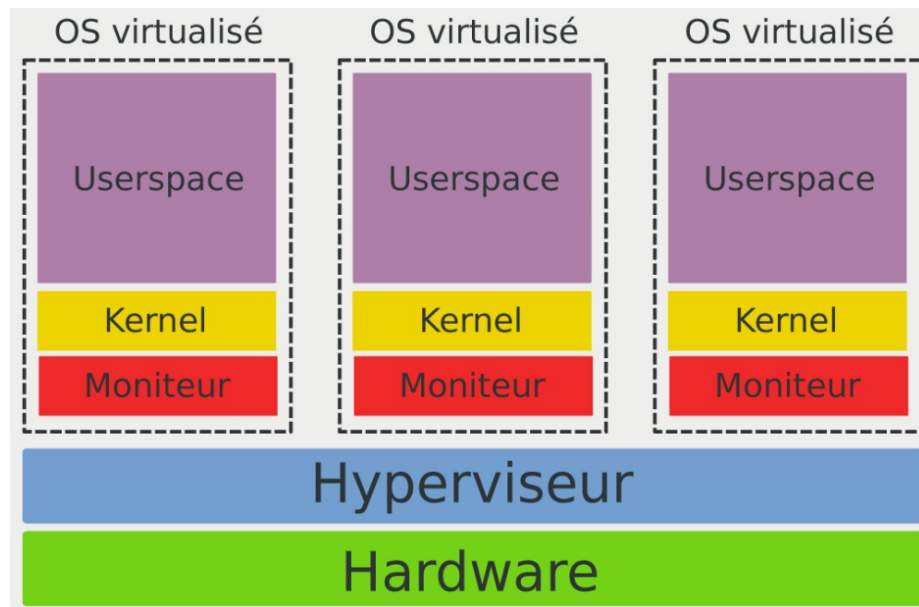


Figure 3 : Hyperviseur de type 1

Un hyperviseur de type 2 – hosted – est un logiciel qui s’exécute au-dessus d’un système d’exploitation hôte. Ce type de virtualisation n’est pas recommandé pour une utilisation serveur. En effet, les systèmes d’exploitation invités doivent traverser deux couches logicielles avant d’accéder au matériel physique, réduisant d’autant les performances. L’avantage de cette solution réside dans sa simplicité de mise en œuvre, c’est une sorte d’émulateur qui permet l’exécution de plusieurs systèmes d’exploitation sans nécessiter de machine dédiée. Il permet la création d’un environnement de développement / tests sans impact sur le système hôte.

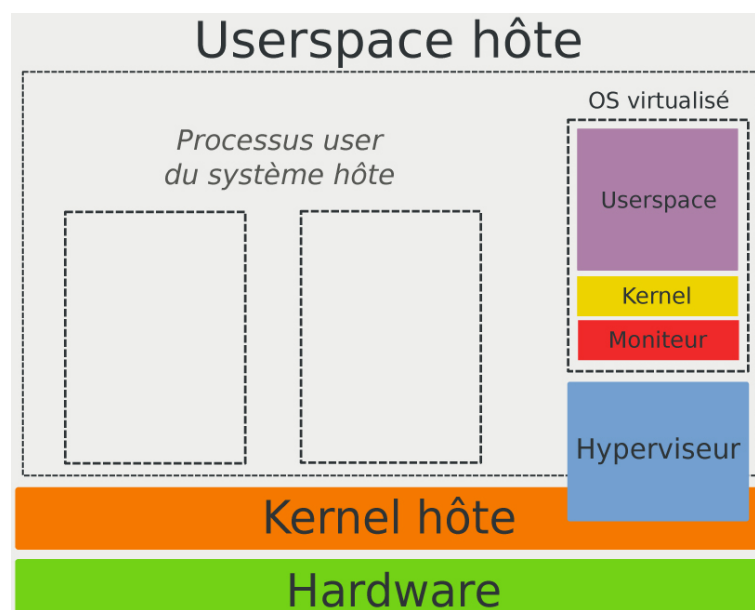


Figure 4 : Hyperviseur de type 2

Les micro-processeurs x86 modernes intègrent des instructions dédiées à la virtualisation : Intel VT-x depuis 2005 et AMD-V depuis 2006. Ces instructions apportent un gain de performance et permettent aux OS invités d'accéder directement aux composants physiques sans passer par l'hyperviseur.

Le présent article traite de la plateforme de virtualisation VMware vSphere, l'hyperviseur de type 1 basé sur l'architecture VMware ESXi. C'est une solution propriétaire dont le prix de départ débute à 1 300€. Il est possible de tester le produit gratuitement pendant 60 jours.

Le management des machines virtuelles n'est pas possible directement depuis la machine physique. Celui-ci est réalisé via le logiciel client VMware vSphere web client – sur la figure 5, la machine virtuelle appelée « vsphere ».

2.2 Avantages & inconvénients

Les avantages de la virtualisation sont nombreux :

- **Flexibilité** de la puissance de calcul : accroître les ressources lors d'événements saisonniers et/ou à des heures de pointe.
- **Utilisation optimale** des ressources, d'autant plus vrai lorsque les services ne sont pas sollicités au même moment : serveur d'applications, serveur de batchs, serveur de sauvegarde, etc. La charge moyenne d'un serveur physique virtualisé est de 35%²⁰.
- **Architecture tolérante aux pannes**
- **Haute disponibilité** des applications et/ou services – **failover**.
- **Administration du parc simplifiée** : le déploiement et la migration des machines virtuelles sont effectuées en quelques clics ou par programmation (API). Dupliquer l'ensemble d'un système est aussi aisé que de copier/coller un fichier ! Les tâches régulières sont automatisées.
- **Liberté** aux développeurs de configurer le système à leur façon sans contrainte liée aux politiques de sécurité et sans outrepasser les droits de chacun.
- **Réduction des coûts** : que ce soit de consommation électrique, d'entretien physique, de compatibilité matérielle ou d'espace occupé dans les data center.
- **Diminution des risques liés** au coût/dimensionnement initial des serveurs lors de la définition de l'architecture d'une application. Elle permet la réalisation du prototype sur un serveur peu onéreux. L'ajout de nouveaux serveurs est transparent et évolue en fonction du besoin réel. Il est possible d'allouer dynamiquement des ressources de stockage, c'est le **Thin Provisioning**²¹.
- **Meilleure gestion des incidents**. Les snapshots²² permettent de revenir à un état passé (possibilité d'inclure la mémoire vive : le système est déjà démarré) en quelques secondes. Ils permettent également de tester de nouvelles configurations sans dégrader le système d'exploitation hôte.

Il existe cependant quelques inconvénients :

²⁰ Livre Blanc Smile : virtualisation et cloud open source

²¹ http://fr.wikipedia.org/wiki/Dynamic_Provisioning

²² http://fr.wikipedia.org/wiki/Instantan%C3%A9_%28informatique%29

- En cas de panne de la machine physique, c'est l'intégralité des espaces virtuels qu'elle contient qui est affectée. Il existe des solutions, un peu plus coûteuses, pour prévenir ce genre d'incidents par le biais du Cloud Computing.
- Une brusque montée en charge d'un serveur virtuel peut également nuire au bon fonctionnement des autres.
- Il existe une forte dégradation des performances si les machines virtuelles sont mal configurées. En cas de manque de mémoire, la machine physique effectue du swap disque – invisible au niveau des machines virtuelles.

3 L'Internet moderne : fort trafic & qualité de service

3.1 Architecture web haute disponibilité avec tolérance des pannes

L'architecture présentée est limitée à un seul serveur physique présent dans un data center. La redondance entre les différents serveurs physiques répartis dans plusieurs data center dépasse le périmètre de la virtualisation, c'est la problématique du Cloud Computing. L'étude se focalise sur l'analyse des logs http. La base de données ainsi que le système de fichier distribué entre les différents frontaux web – SAN²³ – ne sont pas évoqués.

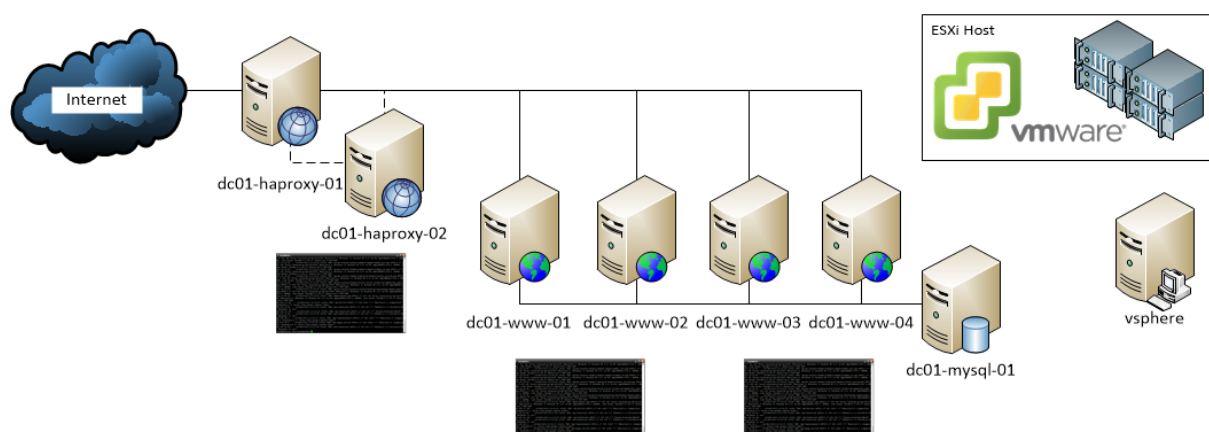


Figure 5 : Architecture web haute disponibilité avec tolérance des pannes

La figure ci-dessus illustre une partie de l'architecture web à scalabilité horizontale. Celle-ci est capable d'absorber un fort trafic ainsi que des pics de charge ponctuelle : la charge est répartie sur les différents frontaux web – « dc01-www-0x ». Ceux-ci forment le pool applicatif web. La répartition de charge est assurée par le load balancer – « dc01-haproxy-0x ». L'algorithme d'ordonnancement des requêtes utilisé est Round-Robin, il s'agit de l'algorithme le plus simple. Les requêtes sont transmises à tour de rôle aux différents serveurs actifs du pool. Les paquets entre le load balancer et les différents frontaux sont routés en utilisant la traduction d'adresse réseaux (NAT, Network Address Translation). Cette technique est simple à mettre en place mais exige que le load balancer assure toute la charge de trafic réseau. Afin de ne pas dégrader les performances, il est important que le load balancer et les serveurs de « calculs » appartiennent au même data center.

La tolérance des pannes est assurée par le load balancer, celui-ci interroge à intervalle régulier (toutes les secondes) les différents frontaux du pool. En cas de défaillance de l'un d'eux – code

²³ Storage Area Network. Solution open source : OpenFiler : <http://www.openfiler.com/products>

d'erreur et/ou temps de réponse qui dépasse une valeur paramétrée – le serveur en question est retiré automatiquement du pool jusqu'à ce que celui-ci soit de nouveau opérationnel.

Le load balancer est un point individuel de défaillance – Single Point Of Failure ou SPOF en anglais. En cas de défaillance de celui-ci, c'est l'ensemble de la plateforme qui devient indisponible. Afin d'assurer la continuité du service, un load balancer secondaire est prêt à prendre immédiatement le relais, c'est le mode « Actif-Passif ». Le trafic Internet entrant arrive sur l'adresse IP virtuelle du load balancer actif. Les deux load balancer s'échangent des battements de cœur – heartbeat – à intervalle régulier afin de s'assurer de leur disponibilité. En cas de panne d'un load balancer, l'adresse IP virtuelle change pour pointer sur le second load balancer qui devient actif le temps de l'indisponibilité.

3.2 Les nouvelles problématiques

Comme vu précédemment, l'ensemble des appels aux ressources d'un serveur web sont historisés dans les fichiers de logs. Plus le trafic du site web est important, plus il devient difficile de trouver l'information pertinente : différencier les logs d'erreur des logs « normaux ».

Les logs sont générés localement au niveau de chaque serveur. Avec la méthode « standard » (cf. chapitre 1.3) il faut connaître le serveur sur lequel s'est produite l'erreur afin de pouvoir se connecter en SSH pour analyser ses logs. Cette opération est de plus en plus chronophage à mesure que le pool de serveur augmente. Il est difficile d'identifier si le problème est présent sur un serveur du pool seul – dysfonctionnement du logiciel de gestion de configuration du serveur esclave – ou bien l'ensemble des serveurs qui composent le pool – problème réseaux, applicatif / bug, etc.

Les portails web, en vue d'accroître la monétisation de leur audience, collectent de plus en plus d'informations personnelles de leurs visiteurs : action, parcours de navigation, partie visible des pages web, etc. Ces informations ne sont pas exploitables si celles-ci sont isolées. Elles délivrent tous leurs potentiels lorsque celles-ci sont recoupées ou indexées.

Les répercussions de l'indisponibilité des systèmes d'informations sont de plus en plus lourdes de conséquences. Sur le plan économique, le manque à gagner peut rapidement devenir conséquent. En 2013, suite à un crash de Google d'une durée de seulement 5 minutes, la perte s'est élevée à 370 000 €²⁴. L'image de marque des sociétés est également touchée car les clients peuvent se demander si la sécurité est suffisante. Enfin, des sites de faible notoriété peuvent perdre des clients.

4 Plateforme d'analyse de log centralisée

4.1 Présentation

L'analyse des logs doit être transparente, celle-ci ne doit pas impacter les services applicatifs. La consommation de ressources CPU, mémoire ainsi qu'I/O pour recueillir les logs doit être la plus faible possible. La plateforme d'analyse de log ne doit pas imposer de contraintes aux différents services applicatifs, celle-ci doit s'adapter aux nombreux et différents formats de log généré. De plus,

²⁴ <http://www.latribune.fr/technos-medias/internet/20130819trib000780722/crash-de-google-28-annees-de-smic-perdues-en-cinq-minutes.html>

elle doit être fiable afin d'éviter les « faux positifs ». Enfin, elle doit être dimensionnée en adéquation avec la volumétrie de l'ensemble des logs afin de garantir une analyse en temps réel.

Il ne s'agit pas d'une solution « clef en main », mais de l'assemblage des produits open source et gratuits Rsyslog, Elasticsearch, Logstash et Kibana. Cela permet une grande flexibilité de configuration afin de mettre au point un produit répondant parfaitement à ses propres besoins. Les technologies employées sont plutôt jeunes, mais bénéficient d'une communauté importante et réactive.



Figure 6 : Chaîne logicielle de la plateforme d'analyse de log en temps réel

4.2 Plus-value

Les logs sont centralisés et indexés dans une base de données distribuée. Les équipes ont un seul outil de statistiques à prendre en main. Ce dernier permet la mise à disposition d'informations techniques à destination d'un large public : service marketing, équipe de développement, équipe système et réseaux. Chaque collaborateur peut créer ses propres tableaux de bord, effectuer des recherches ainsi que des analyses ciblées.

L'analyse d'une volumétrie de données très importante est réalisée en temps réel. Ceci sans agréger ni ajouter les données à intervalle régulier par l'intermédiaire de batchs. Les performances sont linéaires, elles ne se dégradent pas au fur et à mesure que le nombre d'enregistrements augmente. La montée en charge est assurée par scalabilité horizontale.

La collecte d'informations centrée-site – Site-Centric – est passive. Ainsi elle n'alourdit pas la navigation des utilisateurs. Il n'est pas nécessaire d'ajouter des TAGs JavaScript, cela évite de générer des requêtes supplémentaires.

Le formatage ainsi que l'enrichissement des données sont réalisés en temps réel, au même moment que l'indexation. Les plugins d'entrées / sorties ainsi que les filtres sont nombreux et faciles à prendre en main.

4.3 Architecture globale

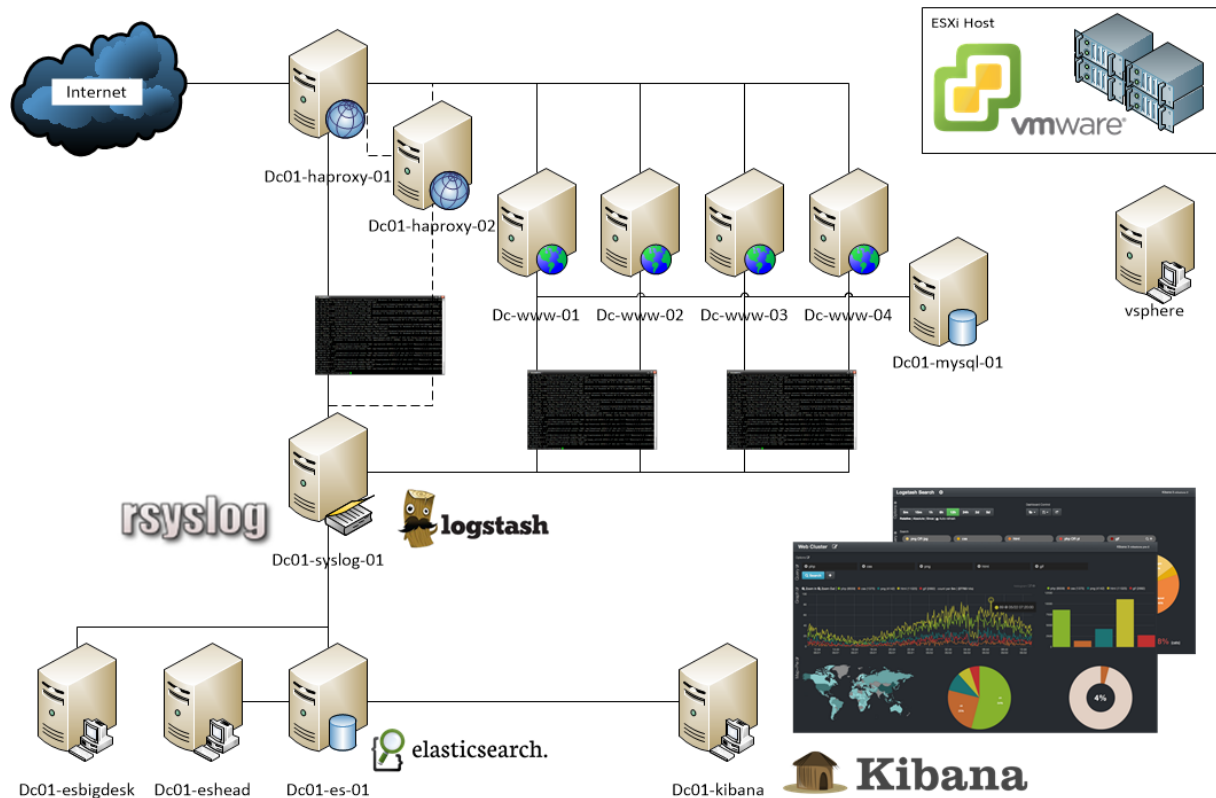


Figure 7 : Architecture web et plateforme d'analyse de log centralisée en temps réel

L'ensemble des machines du pool serveur web ainsi que les load balancer transmettent leurs logs en temps réel au format syslog via TCP à la machine « dc01-syslog-01 ». Celle-ci réalise le formatage et l'enrichissement des logs grâce à Logstash puis stocke le résultat dans la base de données ElasticSearch « dc01-es-01 ». L'analyse des logs par les différents collaborateurs est réalisée via l'interface web Kibana – « dc01-kibana ». Cette dernière requête la base de données ElasticSearch afin de fournir l'ensemble des informations requises pour l'affichage des différents tableaux de bord. Au cas où les traitements réalisés au niveau de l'indexeur « dc01-syslog-01 » représentent un goulot d'étranglement, il est relativement aisé de répartir les calculs sur « n » machines.

4.4 Emission des logs : Apache & autres

4.4.1 Les fondamentaux

L'origine des logs provient des différents services / programmes : Apache, HAProxy, code applicatif du serveur web, etc. Il est préférable de conserver le format par défaut des services standards afin de faciliter la réutilisation et/ou l'intégration d'autres solutions open source. Les logs sont stockés sous forme de document JSON²⁵, c'est-à-dire un document par ligne de log. La base de données n'impose pas de contraintes sur le format de stockage des logs. Cependant, il est important de structurer les informations générées – essentiellement celles des applications personnalisées. Dans le cas contraire, il est difficile d'isoler les différentes informations dans des champs séparés. La tâche de formatage des logs peut être simplifiée et optimisée en générant dès l'origine les logs au format JSON.

²⁵ JavaScript Object Notation : format de données textuelles génériques structurées

Le module « mod_log_config » d'Apache permet une grande flexibilité des informations collectées. Elles sont d'ordre technique : temps de traitement pour servir le contenu (page HTML, image, fichier JavaScript), poids en octets, code http retourné ou encore le protocole utilisé. Il est également possible d'obtenir des informations sur le visiteur qui demande la ressource : adresse IP, url de provenance (referer), navigateur web utilisé (user agent). Pour plus d'informations, se référer à la documentation fournie par Apache²⁶. Ci-dessous, le format de log des accès aux ressources d'Apache.

```
LogFormat "%h %{X-Forwarded-For}i %l %u %t HTTP %{X-Forwarded-Proto}i \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\" t=%D" time_combined_HTTP
```

4.4.2 Problématique liée au load balancer

L'architecture haute disponibilité brouille les informations collectées sur les utilisateurs qui émettent les requêtes. L'adresse IP est celle du load balancer et non celle de l'Internaute. De même, l'utilisation du trafic sécurisé HTTPS est transparent pour les frontaux du pool web. Ces derniers ne fournissent aucun contenu crypté. Les échanges cryptés HTTPS sont uniquement réalisés entre le client et le load balancer. Afin que le navigateur Internet n'émette pas d'alerte de sécurité, l'application web a besoin de savoir lorsque celle-ci est appelée depuis un protocole sécurisé ou non afin d'adapter les appels aux différentes ressources – images, CSS, JavaScript, etc.

Le load balancer rajoute différents headers http pour que les différents frontaux puissent manipuler les vraies données clientes. Ces headers ne sont pas visibles à l'extérieur de la plateforme. Ci-dessous, la configuration des deux headers http évoqués ci-dessus. Pour plus d'informations, consulter la documentation fournie par HAProxy²⁷.

```
# Header X-Forwarded-For
option forwardfor

# Header X-Forwarded-Proto
http-request set-header X-Forwarded-Proto https if { ssl_fc }
http-request set-header X-Forwarded-Proto http if !{ ssl_fc }
```

4.5 Transport des logs : Rsyslog

Le transport des logs, aussi bien local que distant, est réalisé via le protocole syslog - c'est également le format des messages échangés. Ce dernier est le standard de journalisation d'événements sur les systèmes Unix / Linux. De par ce fait, de nombreux services supportent ce protocole qui est fiable, robuste, consomme peu de ressources et permet la mise en tampon des événements. Sa configuration est simplissime. Rsyslog est installé par défaut dans la plupart des distributions Linux. L'identification des événements est réalisée grâce à deux critères (entier définis dans le protocole) :

- L'origine de l'événement – Facility – : 0, « kernel messages » jusqu'à 23, « local7 ».
- Le niveau de gravité – Severity – : du plus grave 0, « Emergency » au moins important 7, « Debug »

²⁶ http://httpd.apache.org/docs/current/fr/mod/mod_log_config.html

²⁷ <http://cbonte.github.io/haproxy-dconv/configuration-1.5.html>

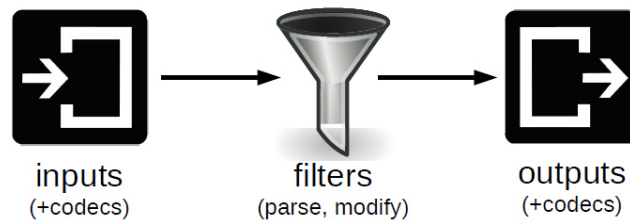


Figure 8 : Principe de fonctionnement d'Rsyslog

La Figure 8 illustre le principe de fonctionnement de Rsyslog qui repose sur l'enchaînement de trois blocs de traitement :

- Plug-in d'entrées – inputs – pour acquérir des informations depuis de multiples sources : pipe, TCP, UDP, etc.
- Des manipulations peuvent être appliquées aux données reçues lors de l'étape précédente : convertir une longue chaîne de caractères en de multiples champs JSON.
- Pour finir, les données sont transmises aux plug-ins de sortie – outputs – afin d'être stockées : fichier, MySQL, ElasticSearch, réseaux, etc.

L'utilisation des filtres Rsyslog est complexe, ils ne sont pas utilisés. Rsyslog fonctionne uniquement en mode relais : les logs émis sont véhiculés localement via un tube de communication – pipe – jusqu'au serveur Rsyslog avant d'être transmis au serveur Logstash via TCP.

Ci-dessous, la configuration d'Apache pour émettre localement des évènements d'origine personnalisées « local7 », dont le niveau de gravité est « info » dans le format précédemment défini « time_combined_HTTP ».

```
CustomLog "|/usr/bin/logger -p local7.info -t 'apache2-access'"
time_combined_HTTP
```

La transmission via TCP est réalisée via la ligne ci-dessous. L'ensemble des évènements d'origine « local7 » (peu importe le niveau de gravité) est transmis au serveur 192.168.1.19 – un seul arobase dans le cas d'une transmission via UDP.

```
local7.* @@192.168.1.19
```

4.6 Formatage & Enrichissement des logs : Logstash

4.6.1 Présentation

Logstash est un outil écrit en Ruby / Java dédié à la collecte, l'analyse et le stockage de log. Celui-ci est distribué sous forme d'un unique « jar ». Le seul prérequis à son fonctionnement est l'installation d'un Java Runtime Environment (JRE)²⁸. Il est supporté officiellement par ElasticSearch²⁹. Son mode opératoire est proche du fonctionnement de Rsyslog : collecte des logs (inputs), multiples traitements opérés sur chaque ligne de log (filters) et stockage (outputs). Logstash supporte les principaux protocoles actuellement utilisés : file, tcp, syslog, email, redis, pipe, ElasticSearch, csv, Solr, RabbitMQ, etc.

²⁸ <http://www.webupd8.org/2012/01/install-oracle-java-jdk-7-in-ubuntu-via.html>

²⁹ <http://www.elasticsearch.org/overview/logstash>

Logstash est nativement scalable horizontalement. Dans une telle architecture, les indexeurs Logstash consomment les événements depuis un cluster Redis³⁰. Le cluster Redis permet de créer une pile FIFO – First In, First Out. Un agent « Logstash shipper » est installé sur chaque frontal du pool web. Ceux-ci se chargent d’insérer les données en queue de liste. Chaque agent « Logstash indexer » lit la tête de la liste de façon bloquante : cela garantit à ce que les événements soient délivrés uniquement à un seul agent et évite de les dupliquer.

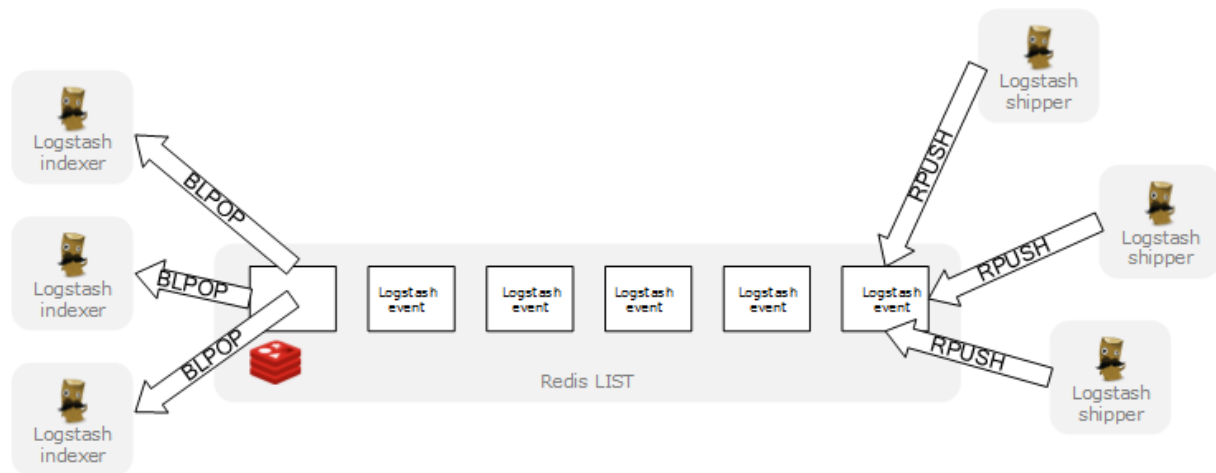


Figure 9 : Architecture Logstash à scalabilité horizontale

La configuration est modulaire via l’ajout de directives – fichiers *.conf – dans le dossier :

```
/etc/logstash/conf.d/*.conf
```

L’ensemble des fichiers sont agrégés par Logstash en vue du paramétrage des trois sections :

- input { ... }
- filter { ... }
- output { ... }

4.6.2 Les plug-ins d’entrées : Input

Suivant la Figure 7, le serveur Rsyslog écoute l’ensemble des messages syslog émis par les différents frontaux web. La configuration est réalisée en quelques lignes. Au démarrage, Logstash démarre un serveur qui écoute sur toutes les adresses IP disponibles sur le port 514 – TCP + UDP.

```
input {
  syslog {
    type      => "syslog-apache2"
    port      => 514
    host      => "0.0.0.0"
  }
}
```

La version de Logstash utilisée (1.3.2) au moment de la rédaction de l’article possède un bug au niveau du protocole syslog : tous les messages ont la même origine et sévérité. Pour différencier les messages, il faut associer un numéro de port différent par service. Le champ « type » permet de

³⁰ Base de données open source NoSQL clefs / valeurs scalable horizontalement : <http://redis.io/>

conditionner les traitements réalisés par les « filters ». Ci-dessous, le paramétrage de l'écoute des logs des load balancer.

```
input {
  syslog {
    type          => "syslog-haproxy"
    port          => 515
    [...]
  }
}
```

4.6.3 Les plug-ins de traitements : Filters

La force de Logstash réside essentiellement dans la simplicité de mise en œuvre et la richesse de ses filtres. Les « tags » et champs « type » permettent le conditionnement et la gestion des erreurs. L'exécution des filtres est le chaînage de l'ensemble des filtres. La sortie des données d'un filtre, fournit les données d'entrées du prochain filtre. En cas de besoin non couvert par le panel de filtres fournis, il est possible de développer ses propres plug-ins en langage Ruby. Cependant, ceux livrés en standard permettent un grand nombre d'opérations telle que la gestion du multilingue des crashes logs, le dédoublonnement ou l'anonymisation des données.

Les RegExp ne sont pas en reste, avec le filtre « Grok » : un ensemble de « méta RegExp ». Celui-ci permet l'écriture et la réutilisation de RegExp de façon étonnement lisible : pas besoin d'être un expert en expression régulière pour écrire des expressions complexes. De plus un débogueur des expressions Grok est disponible en ligne³¹.

```
grok {
  match          => ["message", "%{DAY:d} %{IP:clientip} %{ TIME} [...]"]
  overwrite      => ["message"]
}
```

Dans l'exemple ci-dessus, le filtre « Grok » cherche à extraire deux informations de la chaîne de caractères source « message » : « d », une chaîne de caractères de type « DAY » et « clientip », une adresse IP de type « IP ». Au cas où les données fournies en entrée respectent le format de l'expression « Grok », les données nommées – « d » et « clientip » – sont peuplées de leur valeur. Dans ce cas, Logstash génère autant de nouveaux champs que de valeurs nommées capturées. La valeur « TIME » n'est pas nommée : celle-ci ne sera pas exploitée par Logstash. Dans le cas contraire, si les données analysées ne correspondent pas à l'expression Grok, une erreur sera soulevée par l'ajout d'un tag (nommé par défaut : « _ grokparsefailure »). Le type « IP » est composé des types « IPV6 » et « IPV4 » : IP (?:%{IPV6}|%{IPV4}). Le type « IPV4 » est lui composé uniquement de RegExp « standard » : IPV4 (?<![0-9])(?:(?:25[0-5]|

L'analyse des dates est, elle aussi, puissante. Outre la prise en charge d'un nombre important de formats de dates, les événements peuvent être estampés soit de la date d'entrée du log, soit d'une date contenue dans le log lui-même. Cela permet d'avoir la date réelle de l'événement lorsque les logs sont indexés par intervalle périodique via des batchs.

```
date {
  # [IN] 31/Dec/2013:01:52:07 +0100
  # [OUT] 2013-12-31T01:27:09.000+01:00
}
```

³¹ <http://grokdebug.herokuapp.com/>

```

match          => [ "httpdate" , "dd/MMM/YYYY:HH:mm:ss Z" ]
target         => "@timestamp"
remove_field   => ["timestamp", "httpdate"]
}

```

Il est possible de modifier le nom des champs, ajouter ou supprimer des tags. C'est utile pour conditionner les traitements.

```

mutate {
  replace          => ["proto", "%{XForwardedProto}"]
  uppercase        => ["proto"]
  remove_field     => ["XForwardedProto"]
}

```

Outre le formatage des évènements, il est également possible de les enrichir. Le filtre « geoip » permet, à partir de l'adresse IP contenue dans les logs, de connaître les coordonnées GPS, le pays ainsi que la ville de l'Internaute auteur de la requête. La base de données utilisée est celle fournie par Maxmind³², un leader de la géolocalisation par IP.

```

geoip {
  source          => "clientip"
  database        => "/var/geoip/GeoLiteCity.dat"
}

```

Le filtre « useragent » permet d'isoler les informations qui identifient le navigateur web en une multitude de champs : famille, version majeure, système d'exploitation, etc.

Il est possible de conditionner l'exécution de certains filtres. La syntaxe est similaire à celle utilisée dans la majorité des langages de programmation actuels : « if », « else if », « else ». Les tests peuvent porter sur des valeurs de chaîne de caractères ainsi que des tableaux (surtout utilisés par les « tags »). Les opérateurs de comparaison sont proches de ceux disponibles dans SQL : égalité, RegExp, inclusion, etc. Les conditions peuvent également être appliquées sur les filtres de sorties.

```

if [program] == "apache2-access" {
  [...]
} else if !("_grokparsefailure" in [tags]) {
  [...]
} else {
  [...]
}

```

Il est possible d'ignorer certains évènements en les supprimant de la chaîne des filtres. Ceci afin de ne pas « polluer » la base de données par des évènements parasites, c'est le filtre « drop ».

```

drop { }

```

³² <http://www.maxmind.com/en/home>

4.6.4 Les plug-ins de sorties : Output

Une fois qu'un évènement a parcouru l'intégralité de la chaîne des filtres de traitement et que celui-ci n'a pas été supprimé, il débute la chaîne des filtres de sorties : c'est la même philosophie. Un même évènement peut être transmis vers plusieurs protocoles simultanément : email, redis, fichier, stdout, MongoDB, etc. Ou encore le format de prédilection Elasticsearch. Comme vu avec les filtres de traitements, il est également possible de conditionner les différents filtres de sorties. Dans la plupart des cas, le format de sortie est un document JSON.

Par défaut, la sortie Elasticsearch génère un index dynamique daté au moment de l'opération : `logstash-%{+YYYY.MM.dd}`. C'est également la valeur attendue par Kibana. Le paramètre « `flush_size` » est important, celui-ci permet d'augmenter les performances d'indexation en utilisant l'API « `bulk` » d'ElasticSearch. Plus cette valeur est élevée, plus la mémoire tampon Logstash est importante et plus le nombre de documents indexés dans un même bloc est conséquent.

```
output {
  stdout { debug          => true }
  file { path             => "/var/log/logstash/output.log" }
  elasticsearch { host    => "192.168.1.16" }
}
```

4.7 Stockage des logs : Elasticsearch

4.7.1 Présentation

ElasticSearch est une surcouche du moteur d'indexation libre Lucene³³. Ce dernier se définit comme un framework Java dédié aux problématiques d'indexation et de recherche de contenus. Il est complexe à mettre en place dans sa version originelle. Elasticsearch intègre nativement l'ensemble des prérequis pour l'exploitation d'un index distribué haute disponibilité, scalable horizontalement et tolérant aux pannes. Il est lui aussi écrit en Java. Comme Logstash, il requiert l'installation d'un JRE²⁸ pour fonctionner. Le requêtage d'ElasticSearch est réalisé via l'API REST (port par défaut 9200) par l'échange de documents JSON, aussi bien pour l'interrogation que pour les réponses. La syntaxe des requêtes est celle de Lucene³⁴.

Les évènements émis par Logstash sont de petites tailles, mais très nombreux. Il est important de buffériser l'indexation des documents, sous peine de générer un nombre important de segments (un segment par log). Ceci est préjudiciable, car source de nombreuses opérations de fusions (merge) effectuées par le processus en charge de la ré-indexation de l'index. Un segment est composé d'un ensemble d'évènements (écriture par paquets). Les données placées en cache sont d'abord écrites sur disque, dans un fichier appelé Translog. Ceci permet de ne pas perdre d'informations en cas de défaillance d'ElasticSearch. La reprise sur panne consiste à rejouer le fichier Translog.

4.7.2 Problématiques d'indexation de logs dans un index « full texte »

Les traitements de pré-indexation d'analyse des mots – Analyser – doivent être minimalistes, voir désactivés pour une grande partie des champs. D'une façon générale, l'ensemble des termes qui composent un log sont pertinents et ne doivent pas être modifiés sous peine de perdre ou dénaturer

³³ <http://lucene.apache.org/core/>

³⁴ http://lucene.apache.org/core/3_6_1/queryparsersyntax.html

l'information. L'action du « Tokenizer » est contre-productive, puisqu'elle disperse l'information dans différents termes. L'étude du user agent des navigateurs doit être réalisée ligne par ligne, le numéro de version du navigateur ne doit pas être dissocié du nom du navigateur.

Pour exploiter pleinement Kibana (cf. chapitre 4.8), il est important de définir le schéma (mapping) par défaut des différents champs qui seront utilisés. L'opération consiste principalement à définir les types de champs numériques et désactiver l'analyse des mots (actif par défaut dans Elasticsearch). Sans cela, les requêtes de données numériques calculées (minimale, etc.) peuvent retourner des erreurs. La définition du schéma initial est réalisée manuellement via un appel cURL :

```
curl -XPUT http://192.168.1.16:9200/_template/logstash_index -d '{
  "template" : "logstash-*",
  "settings": {
    "index.cache.field.type": "soft",
    "index.store.compress.stored": true
  },
  "mappings" : {
    "_default_" : {
      "_all" : {"enabled" : false},
      "properties" : {
        "@message":          { "index": "analyzed",    "type": "string" },
        "@timestamp":        { "index": "not_analyzed", "type": "date" },
        "agent":              { "index": "not_analyzed", "type": "string" },
        "bytessent" :         { "index": "not_analyzed", "type": "integer" },
        "clientip":           { "index": "type": "string" },
        [...]
      }
    }
  }
}
```

La modification de la structure de l'index sans interrompre le service est plus complexe, celle-ci doit être réalisée en plusieurs étapes, par l'utilisation d'alias d'index³⁵.

Le type de champ « IP » d'ElasticSearch est une représentation numérique d'une adresse IP. Cela vise à simplifier le classement ainsi que l'interrogation par intervalle. A contrario, il n'est plus possible d'utiliser les outils habituels d'analyse d'adresse IP tels que traceroute ou whois. Afin de conserver le format d'origine, il faut définir un type de champ « String ».

4.8 Analyse des logs temps réel : Kibana

4.8.1 Présentation

Kibana est un produit officiellement supporté par Elasticsearch. Il s'agit d'une application web HTML5 / JavaScript responsive design³⁶ distribuée sous forme d'un zip. Aucune infrastructure n'est requise. La configuration se résume à modifier l'adresse du serveur Elasticsearch à interroger dans le fichier de configuration « config.js » (la valeur par défaut est <http://localhost:9200>). La consultation des tableaux de bord requiert un navigateur web récent qui supporte les instructions HTML5.

³⁵ <http://www.elasticsearch.org/blog/changing-mapping-with-zero-downtime/>

³⁶ La mise en page du contenu des pages web s'adapte aux caractéristiques techniques du navigateur web

Kibana est un Front-end Elasticsearch fortement personnalisable et collaboratif dédié à l'analyse d'évènements. Celui-ci exploite les données d'un serveur / cluster Elasticsearch commun à l'ensemble des départements d'une entreprise. Kibana permet à chaque collaborateur de créer ses propres tableaux de bord afin de focaliser et analyser les données pertinentes à chacun. Ce-dernier peut être partagé et échangé entre les différents services.

4.8.2 Tableau de bord & analyse

Les numéros entre crochets [x] font références aux numéros inscrits en rouge sur la Figure 10.

Avant de détailler le fonctionnement des tableaux de bord Kibana, il est important de comprendre deux notions fondamentales utilisées dans le domaine de l'analyse d'audience (web analytics). Celles-ci sont au cœur des solutions d'analyses tels Google Analytics³⁷, Adobe SiteCatalyst³⁸ ou AT Internet³⁹ (elles sont exprimées en anglais) :

- Les « **Dimensions** » : décrivent les données. Il s'agit d'un attribut descriptif ou caractéristique d'un objet qui peut être composé de plusieurs valeurs. Par exemple, une donnée géographique peut avoir les *dimensions* « Latitude », « Longitude », « Nom de la ville », « Nom du pays », etc. Les valeurs de la *dimension* « Nom du pays » peuvent être « France », « Russie », « Allemagne », etc. Il en est de même pour les données de logs que l'on souhaite analyser : une *dimension* pertinente est le code http⁴⁰ renvoyé par le serveur web en réponse à la requête d'une ressource. Les valeurs de cette *dimension* appelée « response » sont : « 200 », « 301 », « 302 », « 404 », « 500 », etc.
- Les « **Metrics** » : mesurent les données. Ce sont des éléments individuels d'une *dimension* qui peuvent être mesurés, comme une somme ou un ratio. Par exemple, la *dimension* « Nom du pays » peut être associée à une *metric* telle que « Population » qui a pour valeur la somme de tous les résidents du pays en question. Les *metrics* peuvent être « simples » ou calculées à partir de *metrics* « simples ». Par exemple, la *metric* du taux de rebond – bounce rate – est le rapport entre le nombre total de visites et le nombre de visites limitée à une page.

Bien qu'il soit possible d'exploiter les *dimensions* et les *metrics* séparément, elles sont généralement utilisées conjointement. Ce qui donne un sens aux données, ce sont les valeurs des *dimensions* et des *metrics* ainsi que le rapport qui existe entre ces valeurs.

Dans l'exemple ci-dessous, la *dimension* « Pays » peut être associée aux *metrics* « Population » et « Surface ». La *metric ratio* « Densité » est calculée à partir des données des *metrics* « Population » et « Surface » afin d'accroître les informations sur ces différents pays.

<i>Dimension</i>	<i>Metric</i>	<i>Metric</i>	<i>Metric (Ratio ; valeur calculée)</i>
Pays	Population	Surface (en km ²)	Densité de population (hab. / km ²)
France	66 600 000	641 185	103
Allemagne	80 585 700	357 026	225
États-Unis d'Amérique	317 453 000	9 629 048	32

³⁷ <http://www.google.com/analytics/>

³⁸ <http://www.adobe.com/fr/products/sitecatalyst.html>

³⁹ Anciennement XiTi : <http://www.atinternet.com/produits/web-analytics-2/>

⁴⁰ <http://tools.ietf.org/html/rfc2616>

Un tableau de bord Kibana est composé de trois zones :

- **[1]** Une zone de saisie des requêtes Lucene ou RegExp. Chaque requête engendre une source d'analyse : *Dimension*.
- **[2]** Une zone de filtrage des résultats. Elle permet d'imposer des conditions sur les différents champs du flot de données et d'analyser des *metrics* spécifiques, soit d'une valeur précise (exemple : type de log « accès Apache »), soit d'une plage d'intervalle (exemple : événements datés du 1^{er} janvier 2014 au 1 février 2014).
- Une zone d'analyse des résultats interactive : cliquer sur les valeurs des résultats (mono-valeur, multi-valeurs ou intervalle) permet d'ajouter des filtres (cf. zone de filtrage, ci-dessus).

La construction de la partie résultat est réalisée par l'ajout de « bloc type » qui requiert un minimum de configurations : nombre de colonnes utilisées par le composant, champ servant de source de données, agrégation ou non des résultats, valeurs calculées (minimale, moyenne ou maximale). La zone des résultats est matérialisée par une grille composée de 0 à 12 colonne(s) sans limitation du nombre de lignes. Les principaux « blocs type » disponibles sont :

- Histogramme : Par défaut, l'ensemble des *dimensions* saisies dans la zone des requêtes Lucene ou RegExp sont affichées sous forme de courbes **[3]** **[4]**, bâtons ou points. Il est possible de sélectionner les *dimensions* que l'on souhaite analyser par chaque histogramme. Le mode de calcul par défaut est le comptage des événements de chaque *dimension* **[3]**. Il est possible de recouper les données de chaque dimension avec une *metric* de type numérique : calcul de la valeur minimale, maximale, moyenne **[4]** ou somme de toutes les valeurs. Il est possible d'agréger ou non des résultats. La précision des résultats peut varier d'une seconde à une année.
- **[5]** Diagramme camembert : Exploite un champ du flot de données : une *metric*. Les *dimensions* saisies dans la zone des requêtes ne sont pas prises en compte.
- **[6]** **[7]** Tableaux d'événements : affiche l'ensemble des *metrics* pour les *dimensions* sélectionnées (par défaut, toutes). Une sélection de *metrics* est mise en avant **[7]**, l'ensemble des champs disponibles sont affichés en colonne **[6]**. Un clic sur l'un d'entre eux permet de consulter le « Top 10 » des valeurs – cf. Figure 11. Il est possible de filtrer l'ensemble du tableau de bord sur l'une des valeurs en cliquant sur l'icône « loupe » ou « interdit ».
- Carte géographique : USA, Europe, Monde. Utile pour avoir une représentation graphique de la provenance des visiteurs.
- Tendances : hausse, baisse, etc. La granularité de l'échantillon de données analysées pour l'étude des tendances est paramétrable : une heure, une journée, une semaine, etc.
- Requêtes dérivées : Utilise la recherche par facette d'ElasticSearch⁴¹

L'analyse des données est réalisée en temps réel. C'est un avantage qui permet d'avoir une vision réelle d'une campagne dès son lancement. Ceci malgré un volume de données colossal. Sans filtre, l'analyse n'a aucun sens, toutes les informations sont mélangées : logs de production, log de développement, log des switches, log applicatifs, etc. La simplicité d'utilisation de Kibana est le reflet de la qualité des traitements effectués en amont, essentiellement via les filtres de traitements de Logstash.

⁴¹ <http://dev.david.pilato.fr/?p=148>

Il est possible de tester en « live » cette solution sur le site de l'éditeur⁴².

Ci-dessous, afin d'illustrer cet article, un bref aperçu de l'analyse des logs d'accès Apache via un tableau de bord Kibana.

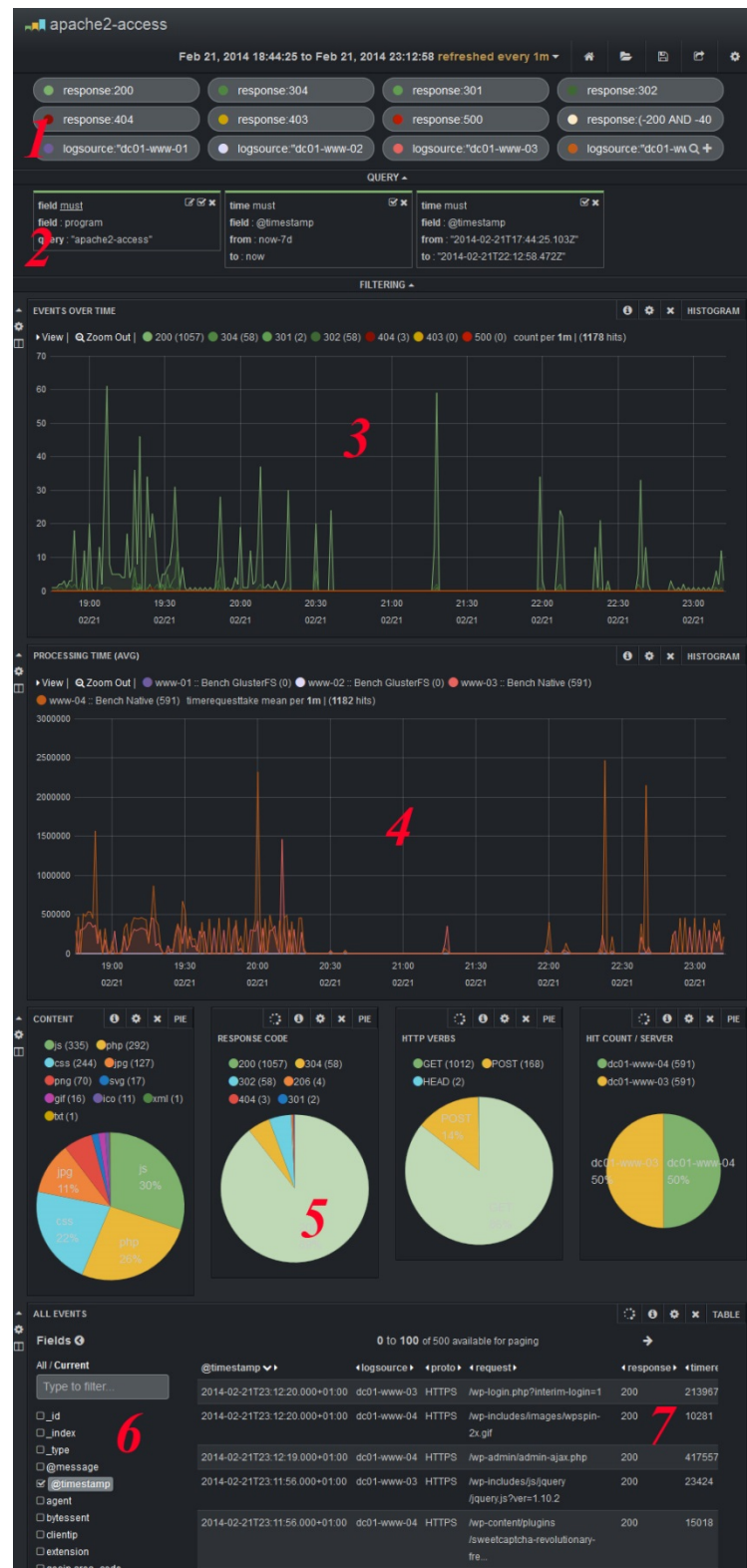


Figure 10 : Visualisation des logs d'accès Apache sur la plateforme Kibana

⁴² <http://demo.kibana.org/#/dashboard>

Ci-dessous, Figure 11, Le « Top 10 » de la metric « geoip.city_name » :

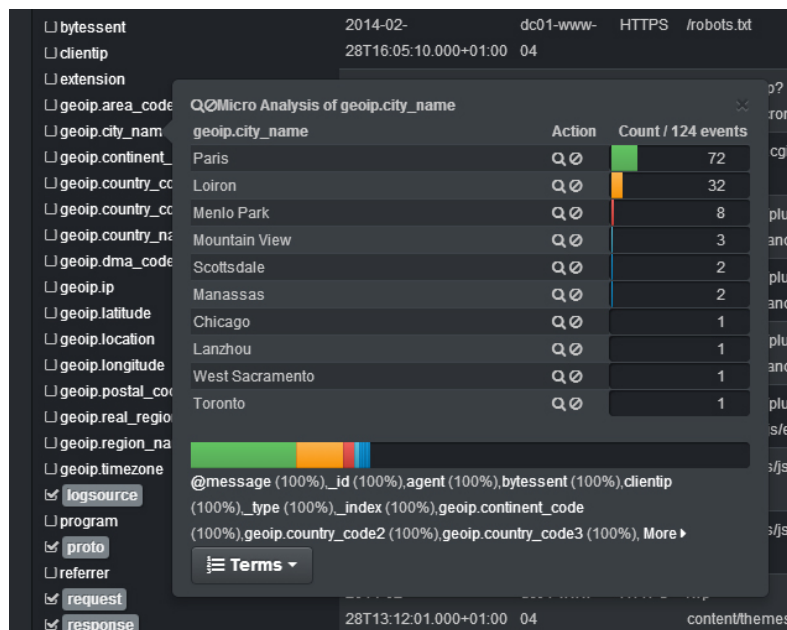


Figure 11 : Top 10 des valeurs d'une metric

Ci-dessous, le même log d'accès Apache affiché premièrement dans son format natif...

```
192.168.1.48 157.56.92.160 - - [27/Jan/2014:09:42:11 +0100] HTTP
http "GET /robots.txt HTTP/1.1" 200 329 "-" "Mozilla/5.0
(compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)" t=3495
```

... puis via l'interface Kibana. La colonne « action » permet de filtrer les résultats : correspondance (icône « loupe ») ou NON correspondance (icône « interdit »).

2014-01-28T07:31:42.000+01:00

dc01-www-02

HTTP

/robots.txt

200

1234

View: **Table** / JSON / Raw

Field	Action	Value
@message		192.168.1.48 157.56.93.93 - - [28/Jan/2014:07:31:42 +0100] HTTP http "GET /robots.txt HTTP/1.1" 200 329 "-" "Mozilla/5.0 (compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)" t=1234
@timestamp		2014-01-28T07:31:42.000+01:00
_id		2SdvQYBRQPKHFWj8cy2JBQ
_index		logstash-2014.01.28
_type		logs
agent		Mozilla/5.0 (compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)
bytessent		329
clientip		157.56.93.93
extension		txt
geoip.area_code		425
geoip.city_name		Redmond
geoip.continent_code		NA

Figure 12 : Détail d'un log d'accès Apache sur la plateforme Kibana 1 de 2

geoip.country_code2	Q 0 iii	US
geoip.country_code3	Q 0 iii	USA
geoip.country_name	Q 0 iii	United States
geoip.dma_code	Q 0 iii	819
geoip.ip	Q 0 iii	157.56.93.93
geoip.latitude	Q 0 iii	47.68010000000001
geoip.location	Q 0 iii	-122.1206,47.68010000000001
geoip.longitude	Q 0 iii	-122.1206
geoip.postal_code	Q 0 iii	98052
geoip.real_region_name	Q 0 iii	Washington
geoip.region_name	Q 0 iii	WA
geoip.timezone	Q 0 iii	America/Los_Angeles
logsource	Q 0 iii	dc01-www-02
program	Q 0 iii	apache2-access

Figure 13 : Détail d'un log d'accès Apache sur la plateforme Kibana 2 de 2

Conclusion

L'Internet d'aujourd'hui n'a plus grand-chose à voir avec celui d'il y a plus de 20 ans. Fini le temps où celui-ci est développé par des « geeks » pour des « geeks » ! Un véritable écosystème s'est créé, Internet s'est industrialisé et démocratisé. Des outils au préalable réservés à une élite sont maintenant accessibles et utilisés par le grand public. L'utilisateur lambda utilise quotidiennement des dizaines de services Internet en toute simplicité (achat sur Internet, virement en ligne, recherche d'informations, musique et vidéo en streaming, réseaux sociaux, infogérance, etc), sans avoir conscience de la complexité ni des contraintes des systèmes sous-jacents.

La virtualisation apporte un réel gain économique et une réactivité accrue. Le test de prototype grandeur nature peut être réalisé en quelques heures sans acheter de nouveaux serveurs quelle que soit la technologie en jeu. L'allocation dynamique de ressources serveurs afin de gérer des pics ponctuels de trafic par programmation permet d'optimiser l'utilisation des serveurs. Le Cloud Computing, qui repose sur la virtualisation, permet à de petites sociétés de démarrer leurs activités sans investir des sommes importantes dans l'achat de serveurs physiques, en payant l'utilisation réelle des services.

Les services Internet sont de plus en plus complexes à développer à cause des problématiques de fort trafic, de la multitude des terminaux clients, de l'accès Internet mobile, l'offre de service ouverte à l'international, etc. Les répercussions d'une défaillance technique sont de plus en plus fortes, c'est l'image de marque qui est touchée, avec pour conséquence des pertes financières importantes ainsi que la perte potentielle de clients. L'exploitation et la maintenance d'un système d'information fiable et performant est de nos jours (pratiquement) impossible sans outil adapté. On ne peut pas se reposer uniquement sur la réactivité des équipes techniques. La mise en place au sein d'une entreprise de la plateforme d'analyse de log centralisée en temps réel basée sur Rsyslog, Logstash, Elasticsearch, Kibana représente un investissement à forte valeur ajoutée. Celle-ci permet d'automatiser une partie des tâches répétitives, et ainsi, de faire gagner quotidiennement un temps précieux aux équipes. Les données techniques ne sont plus réservées uniquement aux administrateurs systèmes et réseaux, mais aux différents acteurs de l'entreprise. La flexibilité de la solution permet à tout un chacun de créer et partager en deux temps trois mouvements des tableaux de bord correspondant à ses propres besoins.

Les technologies évoquées au travers de cet article sont encore jeunes, en plein essor et bénéficient de nombreuses innovations. Le déploiement de la plateforme d'analyse de logs est modulaire en fonction de la volumétrie que l'on souhaite analyser. L'ensemble des composants qui forment la chaîne d'analyse est scalable horizontalement. De plus, celle-ci est composée uniquement de solutions open source, il n'y a donc aucun coût de licence à prévoir. Enfin la plupart des services et/ou protocoles actuels sont supportés.

Une telle plateforme a été installée par l'hébergeur Cloud Computing RackSpace pour analyser les logs de son service « Mailgun ». Chaque jour, **40 à 60 millions d'évènements de logs sont traités** et stockés pendant 30 jours⁴³.

⁴³elasticsearch.org/blog/using-elasticsearch-and-logstash-to-serve-billions-of-searchable-events-for-customers